

FIAP

MBA em Inteligência Artificial

Redes Neurais Recorrentes

RNN — Recurrent Neural Networks

Conteúdo

 O que é RNN

 Como funciona

 Quando usar

 Limitações

 LSTM & GRU

 Ativação

 Loss & Métricas

 Otimizadores

 Aplicação Python



O que é uma RNN?

Rede Neural Recorrente — feita para dados com ordem e dependência temporal

RNN (Rede Neural Recorrente)

é um tipo de rede neural projetada para processar dados sequenciais — onde a **ordem importa**.

Diferente de uma MLP:

- ✦ Não olha apenas o dado atual
- ✦ Considera o que aconteceu antes
- ✦ Possui memória (hidden state)

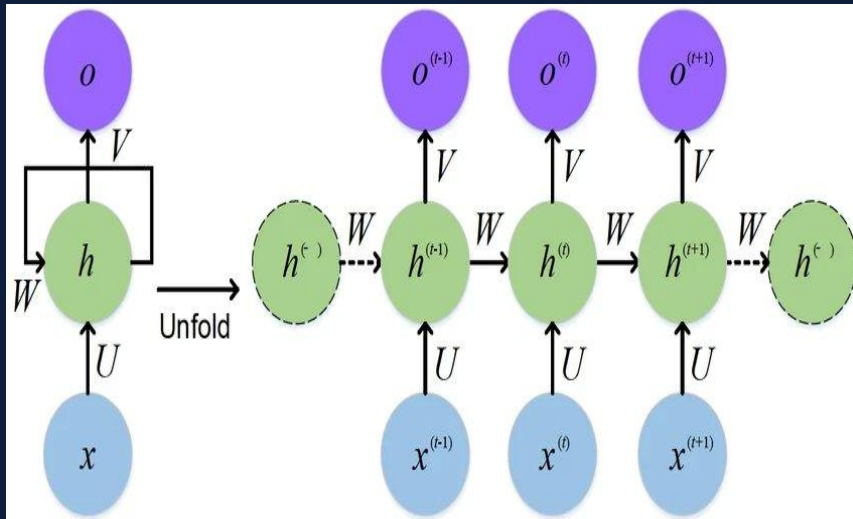
Exemplos de sequências:

texto • fala • sensores
preços • séries temporais



A RNN aprende padrões ao longo do tempo

RNN Desenrolada no Tempo (Unfolded)



A mesma célula se repete no tempo, passando $h_{(t)}$ para o próximo passo → memória temporal



Hidden State (h_t) = memória que flui de passo em passo



Como a RNN Funciona

Processamento sequencial — combinando entrada atual com memória anterior

$$h_t = f (Wx \cdot x_t + Wh \cdot h_{(t-1)} + b)$$

 x_t

Entrada atual
(dato do passo t)

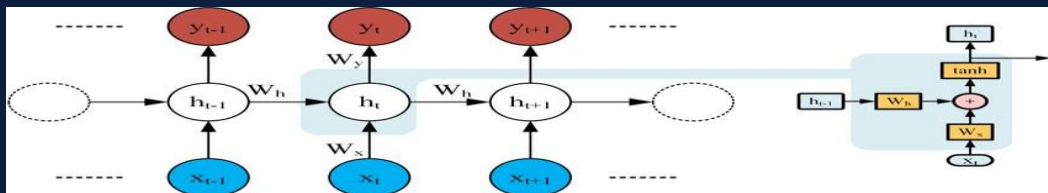
 $h_{(t-1)}$

Memória anterior
(estado oculto)

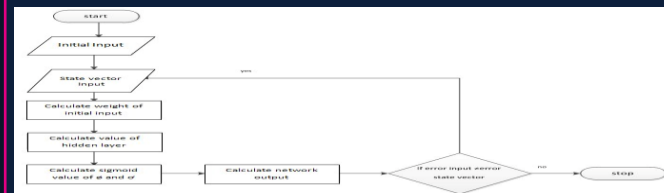
 h_t

Nova memória
(resumo atualizado)

Detalhe: cálculo interno da célula RNN



Fluxo de processamento

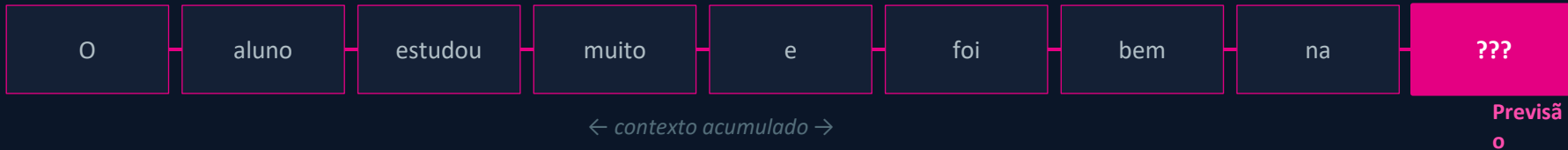




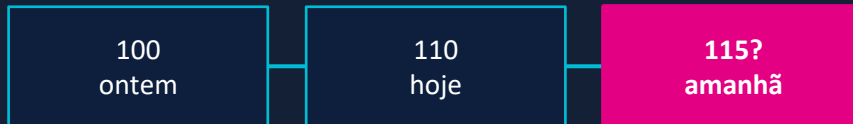
Intuição: Contexto é tudo!

Por que a ordem dos dados importa para prever o futuro?

"O aluno estudou muito e foi bem na ..."



Séries Temporais — a mesma lógica:



A RNN lembra o contexto anterior para tomar decisões melhores a cada passo



Quando Utilizar RNN

Regra de ouro: se a ORDEM dos dados importa → RNN é candidata



Séries Temporais

- ✦ Previsão de vendas
- ✦ Temperatura / Clima
- ✦ Consumo de energia



Texto (NLP)

- ✦ Próxima palavra
- ✦ Análise de sentimento
- ✦ Tradução automática



Áudio

- ✦ Reconhecimento de fala
- ✦ Síntese de voz
- ✦ Detecção de emoção



Sensores / IoT

- ✦ Detecção de falhas
- ✦ Monitoramento contínuo
- ✦ Manutenção preditiva



Aplicações reais: forecasting de demanda, detecção de fraude em sequência, chatbots, análise de logs



Limitações da RNN Simples

O que acontece com sequências muito longas?



Vanishing Gradient

O gradiente diminui exponencialmente a cada backprop ao longo do tempo. A rede 'esquece' eventos distantes e não consegue aprender dependências de longo prazo.



Exploding Gradient

O gradiente cresce de forma exponencial. Os pesos explodem, tornando o treinamento completamente instável e os parâmetros incontroláveis.

Memória ao longo do tempo →

t-8

t-7

t-6

t-5

t-4

t-3

t-2

t-1

t-0



Solução: LSTM (Long Short-Term Memory) e GRU (Gated Recurrent Unit)



Tipos de Arquiteturas RNN

Da mais simples à mais sofisticada

SimpleRNN

✓ Vantagens

- Simples de implementar
- Ideal para ensino
- Boa para seq. curtas

⚠ Limitações

- Vanishing gradient
- Esquece passado distante

📦 Didático / protótipos

LSTM

✓ Vantagens

- Portas: input, forget, output
- Excelente memória longa
- Padrão da indústria

⚠ Limitações

- Mais parâmetros
- Treinamento mais lento

📦 Séries longas / NLP

GRU

✓ Vantagens

- Versão simplificada da LSTM
- Menos parâmetros
- Treinamento mais rápido

⚠ Limitações

- Ligeiramente menos expressivo

📦 Performance vs velocidade

LSTM: portas de controle de memória (σ = sigmoid controla o fluxo, $\tanh$ cria novos valores)





Funções de Ativação em RNN

Qual usar, quando e por quê?

◆ Tanh

Saída: $[-1, 1]$

Controla explosão de valores. Mantém estabilidade numérica. Funciona muito bem com dados sequenciais. Padrão do SimpleRNN.

✓ SimpleRNN → sempre use Tanh

◆ ReLU

Saída: $[0, +\infty)$

Mais rápida computacionalmente. Pode causar instabilidade em RNN pura — o exploding gradient não é controlado naturalmente.

⚠ Evitar em RNN básica

◆ Sigmoid

Saída: $[0, 1]$

Ideal para controle binário de portas (gates). Usada internamente em LSTM e GRU para decidir o que lembrar e esquecer.

✓ LSTM/GRU (automático)

🧠 **Dica:** SimpleRNN → use tanh | LSTM/GRU → usam sigmoid + tanh internamente (automático no Keras)

Tanh: -1 — 0 — $+1$

ReLU: 0 — $+\infty$

Sigmoid: 0 — 0.5 — 1



Função de Perda e Métricas

Como avaliar modelos de séries temporais?

Função de Perda

Regressão

MSE (recomendado)
MAE

Classificação

Binary Crossentropy
Categorical Crossentropy

✓ Regressão → MSE

MAE

Mean Absolute Error

Erro absoluto médio.
Fica na unidade do problema. Fácil de interpretar.
Ex: MAE = 8 → modelo erra ±8 unidades em média.

RMSE

Root Mean Squared Error

Raiz do MSE. Volta à unidade original.
Penaliza mais os erros grandes. Muito usada quando grandes desvios são críticos.

MAPE

Mean Absolute % Error

Erro percentual médio. Útil para o negócio: 'o modelo erra X% em média'.
Cuidado: instável quando valores são próximos de zero.

R²

Coef. de Determinação

Variabilidade explicada pelo modelo.
Complementar — MAE e RMSE costumam ser mais importantes na prática.



Melhor combinação prática: MAE + RMSE (adicione MAPE se o negócio pensar em percentual)

Otimizadores

Qual algoritmo usar e quando?

Adam

✓ Melhor escolha inicial

Combina momentum + adaptação do learning rate. Converte rápido, estável e robusto. Exige menos ajuste manual — ideal para começar.

✓ Vantagens

- Converte rápido
- Lida bem com gradientes ruidosos
- Estável e robusto
- Menos ajuste manual

👉 Sempre — especialmente projetos iniciais

RMSprop

🔄 Boa alternativa

Otimizador adaptativo historicamente bom para dados sequenciais. Pode competir com Adam em RNN/LSTM — vale testar.

✓ Vantagens

- Bom para RNN/LSTM
- Lida com variações no gradiente
- Competitivo com Adam

👉 Quando Adam oscilar ou para comparação

SGD

🔧 Ajuste fino

Método clássico. Pode generalizar melhor no longo prazo, mas exige mais cuidado com learning rate e momentum configurados.

✓ Vantagens

- Pode generalizar melhor
- Controle total do lr
- Muito bem estudado

👉 Após boa base — ajuste fino avançado

```
optimizer = Adam(learning_rate=0.001) # Recomendado para começar
```



Aplicação em Python — Séries Temporais

Previsão com SimpleRNN + Keras

Janela temporal (sliding window)

Entrada: [10, 12, 13, 15, 18] → Saída: 20
Entrada: [12, 13, 15, 18, 20] → Saída: 21

Normalização com MinMaxScaler

```
scaler = MinMaxScaler()  
X = scaler.fit_transform(serie) # → [0, 1]
```

```
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import SimpleRNN, Dense  
from tensorflow.keras.optimizers import Adam  
  
model = Sequential([  
    SimpleRNN(16, activation='tanh', input_shape=(janela, 1)),  
    Dense(1)  
)  
model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')
```

SimpleRNN(16)

16 neurônios recorrentes.
Aprende o padrão temporal.

activation='tanh'

Valores em [-1,1].
Padrão para RNN simples.

Dense(1)

Saída: um valor numérico
(próximo ponto da série).

loss='mse'

Penaliza erros grandes.
Padrão para regressão.



Avaliação do Modelo na Prática

Calculando e interpretando as métricas

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_real, y_pred)
rmse = np.sqrt(mean_squared_error(y_real, y_pred))
mape = np.mean(np.abs((y_real - y_pred) / y_real)) * 100
r2 = r2_score(y_real, y_pred)
```

Métrica	Interpretação	Vantagem	Cuidado
MAE	Erro médio na unidade do problema	Fácil de interpretar	Não penaliza erros grandes
RMSE	Erro na unidade original (raiz do MSE)	Sensível a grandes desvios	Menos intuitivo que MAE
MAPE	Erro percentual médio (%)	Linguagem de negócio	Instável se valores ≈ 0
R^2	% da variância explicada [0–1]	Indica poder explicativo	Menos prático em séries



Próximos passos: SimpleRNN → LSTM | Adicionar EarlyStopping | Testar Adam vs RMSprop | Ajustar janelas



Configuração Recomendada para Aula

Stack ideal para notebook didático de séries temporais



Modelo

SimpleRNN



Ativação

tanh



Loss

MSE



Métricas

MAE + RMSE



Otimizador

Adam
lr=0.001

```
model = Sequential([ SimpleRNN(16, activation='tanh', input_shape=(janela, 1)), Dense(1) ])  
model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')  
model.fit(X_train, y_train, epochs=50, validation_split=0.1)
```



Resumo Final

Tudo que você precisa saber sobre RNN em uma página

O que é

- ✦ Rede para dados sequenciais
- ✦ Memória via hidden state h_t
- ✦ Aprende dependência temporal

Quando usar

- ✦ Séries temporais
- ✦ Texto (NLP) e Áudio
- ✦ Sensores e IoT

Limitações

- ✦ Vanishing gradient
- ✦ Esquece longo prazo
- ✦ Solução: LSTM / GRU

Stack ideal

- ✦ SimpleRNN + tanh
- ✦ Loss: MSE
- ✦ Adam | MAE + RMSE